

MongoDB Three-Laptop Replica Set Setup

Once all 3 laptops can **ping each other over the hotspot**, do the setup in this order. Don't jump straight to experiments yet.

1. Record each laptop's IPv4 address

On each laptop:

```
ipconfig
```

Under **Wireless LAN adapter Wi-Fi**, write down the IPv4 addresses, for example:

```
Laptop 1: 172.20.10.2
Laptop 2: 172.20.10.3
Laptop 3: 172.20.10.4
```

These addresses will be used in the MongoDB replica-set configuration.

2. Install Docker Desktop on all 3 laptops

Install Docker Desktop on all 3 laptops if it is not already installed. After installation, confirm Docker works:

```
docker --version
```

and:

```
docker run hello-world
```

Do this on every laptop.

3. Open MongoDB port 27017 in Windows Firewall

Run PowerShell as Administrator:

```
New-NetFirewallRule -DisplayName "Allow MongoDB from Local Subnet" -Protocol TCP -
    LocalPort 27017 -Direction Inbound -RemoteAddress LocalSubnet -Action Allow
```

This allows MongoDB traffic from devices on the same hotspot subnet.

4. Start one MongoDB container on each laptop

Use the same MongoDB version on all three.

On Laptop 1:

```
docker run -d --name mongo1 -p 27017:27017 -v mongo1_data:/data/db mongo:8.0 --replSet
rs0 --bind_ip_all
```

Laptop 2:

```
docker run -d --name mongo2 -p 27017:27017 -v mongo2_data:/data/db mongo:8.0 --replSet
rs0 --bind_ip_all
```

Laptop 3:

```
docker run -d --name mongo3 -p 27017:27017 -v mongo3_data:/data/db mongo:8.0 --replSet
rs0 --bind_ip_all
```

Then confirm each container is running:

```
docker ps
```

You should see one MongoDB container on each laptop.

5. Test MongoDB port connectivity between laptops

Now that MongoDB is actually running, test TCP port 27017.

From Laptop 1:

```
Test-NetConnection 172.20.10.3 -Port 27017
Test-NetConnection 172.20.10.4 -Port 27017
```

From Laptop 2:

```
Test-NetConnection 172.20.10.2 -Port 27017
Test-NetConnection 172.20.10.4 -Port 27017
```

From Laptop 3:

```
Test-NetConnection 172.20.10.2 -Port 27017
Test-NetConnection 172.20.10.3 -Port 27017
```

You want:

```
TcpTestSucceeded : True
```

in all directions.

6. Enter MongoDB shell on Laptop 1

```
docker exec -it mongo1 mongosh
```

You should get a MongoDB shell prompt.

7. Create the 3-node replica set

Assuming:

```
Laptop 1: 172.20.10.2
Laptop 2: 172.20.10.3
Laptop 3: 172.20.10.4
```

run inside mongosh:

```
rs.initiate({
  _id: "rs0",
  members: [
    { _id: 0, host: "172.20.10.2:27017", priority: 2 },
    { _id: 1, host: "172.20.10.3:27017", priority: 1 },
    { _id: 2, host: "172.20.10.4:27017", priority: 1 }
  ]
})
```

Wait around 10–30 seconds, then:

```
rs.status()
```

You want to see one member as:

```
PRIMARY
```

and two as:

```
SECONDARY
```

8. Confirm replication works

On the primary, create a test database:

```
use dsa5208
```

Insert something:

```
db.test.insertOne({name: "replication_test", value: 100})
```

Then on Laptop 2:

```
docker exec -it mongo2 mongosh
```

Inside mongosh, first allow reading from the secondary for this test:

```
db.getMongo().setReadPref("secondary")
```

Then:

```
use dsa5208
db.test.find()
```

You should see the document inserted from Laptop 1.

Do the same on Laptop 3.

At this point you have successfully built the distributed database:



9. Test failover before doing consistency experiments

On the laptop currently acting as primary:

```
docker stop mongo1
```

Wait a short while, then on Laptop 2:

```
docker exec -it mongo2 mongosh
```

and run:

```
rs.status()
```

One of the remaining nodes should become:

```
PRIMARY
```

Then restart Laptop 1's MongoDB:

```
docker start mongo1
```

It should rejoin as a secondary.

This proves your 3-node replica set and election mechanism are working.

10. Only after that, install Python/PyMongo for experiments

On whichever laptop will run your workload:

```
pip install pymongo pandas matplotlib
```

Then your connection string will be approximately:

```
mongodb://172.20.10.2:27017,172.20.10.3:27017,172.20.10.4:27017/?replicaSet=rs0
```

From there you can start the actual project experiments:

```

Experiment 1
w:1 + local
vs
majority + majority

Experiment 2
primary reads
  
```

```
vs  
secondary reads
```

```
Experiment 3  
causal consistency  
on vs off
```

```
Experiment 4  
primary failure and election
```

For now, I would stop after **Step 8** and make sure replication is completely reliable before writing any experiment code. The immediate next milestone is:

```
Ping works  
-> Docker works on all 3  
-> MongoDB runs on all 3  
-> TCP 27017 works  
-> rs.initiate()  
-> 1 PRIMARY + 2 SECONDARY  
-> replicated document visible
```

Once you reach that state, the infrastructure part of the project is essentially complete.